Tutorial 2.1
Introduction to Classes and Objects

Objectives

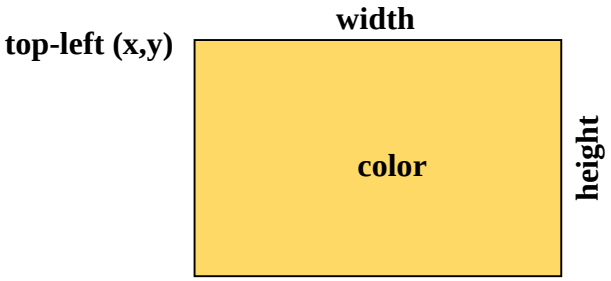
After completing this tutorial, you should be able to

- define classes and create objects
- understand the use of constructor, accessor and mutator methods
- separate class specifications and implementations into files

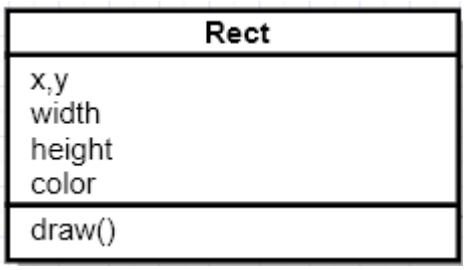
Problem

In this tutorial, you will be using OOP approach to write a program that displays a rectangle moving from one side of the screen to the other side.

How to represent a rectangle?



Class diagram



Task 1: Define Class using Inline vs. Separation styles

Inline: class definition are written inside the class declaration

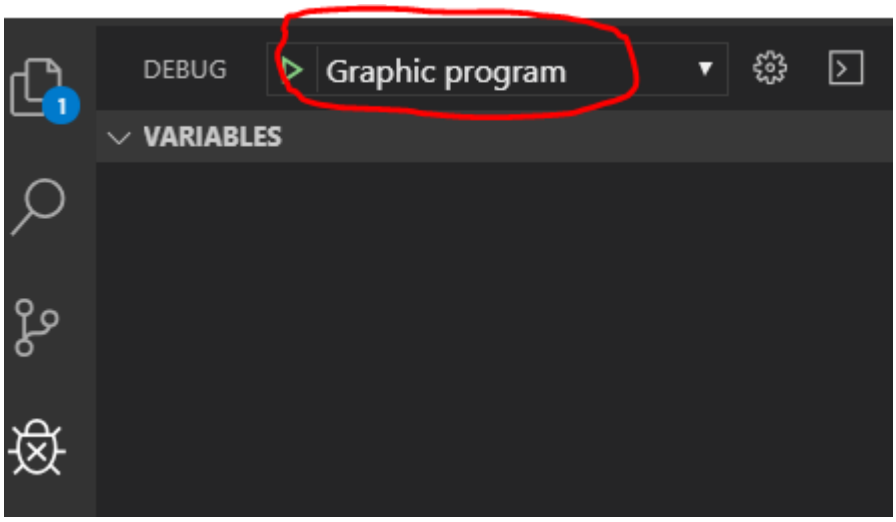
```
class Rect
{
private:
    int x, y; // The top-left corner coordinates
    int width, height;
    int color;

public:
    Rect(int _x, int _y, int _width, int _height, int _color)
    {
        x = _x;
        y = _y;
        width = _width;
        height = _height;
        color = _color;
    }

    void draw()
    {
        setfillstyle(SOLID_FILL, color);
        bar(x, y, x + width, y + height);
    }
};
```

Separation: class declaration and definition are written in separate places.

```
// Class declaration
class Rect
{
private:
    int x, y; // The top-left corner coordinates
    int width, height;
    int color;

public:
    Rect(int _x, int _y, int _width, int _height, int _color);
    void draw();
};

int main()
{
…..
}
// Class definition
```

```
Rect::Rect(int _x, int _y, int _width, int _height, int _color)
{
    x = _x;
    y = _y;
    width = _width;
    height = _height;
    color = _color;
}

void Rect::draw()
{
    setfillstyle(SOLID_FILL, color);
    bar(x, y, x + width, y + height);
}
```

To run the program, choose debug option: **Graphic program**



Task 2: Separate Class Specification and Implementation into Different Files

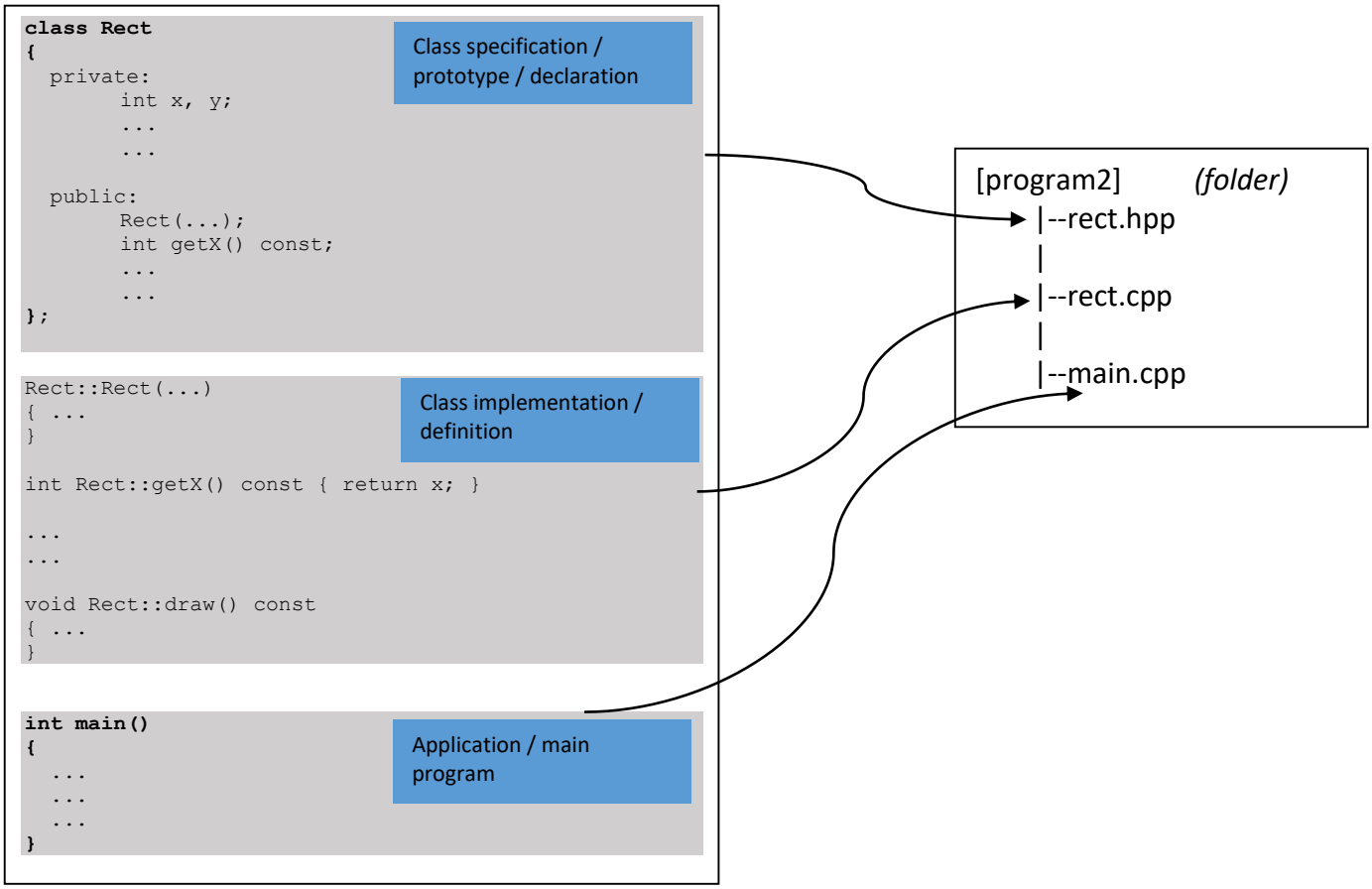
In this section, you will be rearranging program1.cpp into several files. By using this approach, your program now no longer resides in a single .cpp file. Instead, you will be using a folder for each program.

For each class in your program will have to use two files,

- a header file (.hpp or .h) contains the class declaration
- a code file (.cpp) contains the class definition

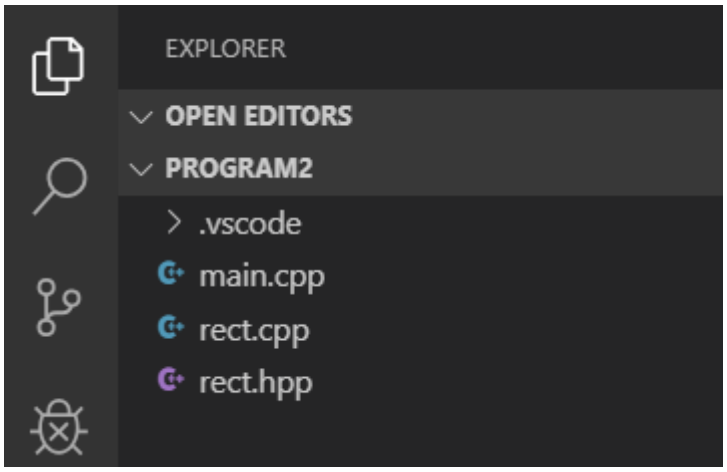
by convention the file name follows the class name with snake case. For example, **rect.hpp** and **rect.cpp**, respectively.

The main function will reside in its own file, e.g, **main.cpp**

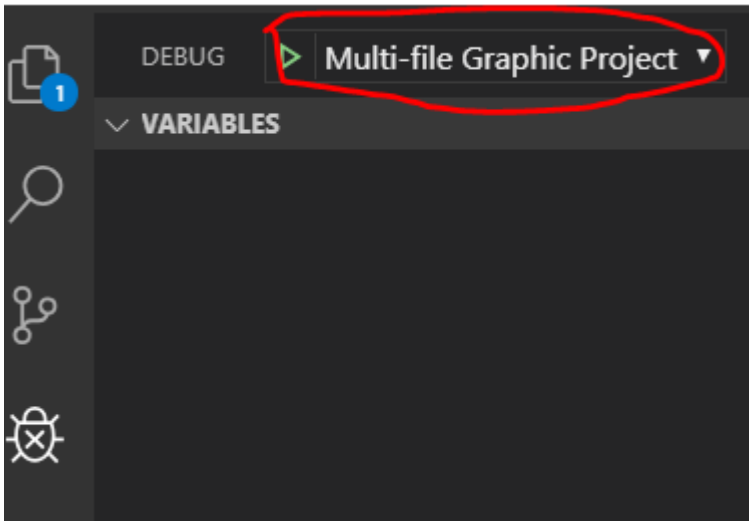


Follow along my demo to rearrange the program into separate files.

Your new program folder should look like this:



To run the program, choose debug option: **Multi-file Graphic Project**



Task 3: Add Other Constructors (Overloading Constructors)

Files to edit: **rect.hpp** and **rect.cpp**. Follow along my demo.

```
class Rect
{
private:
    int x, y; // The top-left corner coordinates
    int width, height;
    int color;
public:
    Rect(int _x, int _y, int _width, int _height, int _color); // normal constructor
    Rect(int _x, int _y, int _length, int _color); // for creating a square
    Rect(); // default constructor
    …..
};
```

Task 4: Add Accessors and Mutators to the Class

Files to edit: **rect.hpp** and **rect.cpp**. Follow along my demo.

```
class Rect
{
private:
    int x, y; // The top-left corner coordinates
    int width, height;
    int color;

ublic:
    Rect(int _x, int _y, int _width, int _height, int _color); // normal constructor
    Rect(int _x, int _y, int _length, int _color); // for creating a square
    Rect(); // default constructor

    int getX() const;
    int getY() const;
    int getWidth() const;
    int getHeight() const;
    int getColor() const;

    void setX(int value);
    void setY(int value);
    void setWidth(int value);
    void setHeight(int value);
    void setColor(int value);

    void draw() const;
    void undraw() const;
};
```

Task 5: Add Simple Animation

File to edit: **main.cpp**. Follow along my demo.

```
int main()
{
    int screenWidth = getmaxwidth();
    int screenHeight = getmaxheight();
    initwindow(screenWidth, screenHeight, "Tutorial 2");

    Rect r(300, 300, 200, 250, COLOR(0, 0, 255));
    Rect s(0, 0, 150, 150, YELLOW);

    r.draw();
    s.draw();
    getch();

    s.undraw();
    getch();

    int dx = 50;
    while (!kbhit())
    {
        s.draw();
        delay(100);
        s.undraw();
        int x = s.getX();
        s.setX(x + dx);

        if (
            (s.getX() <= 0) ||
            ((s.getX() + s.getWidth()) >= screenWidth))
            dx = -dx; // invert direction. but the speed remains
    }

    getch();

    return 0;
}
```

Task 6: Refactor Code

Code Refactoring #1: draw() and undraw()

The following code are **WET** (Write Everything Twice)

```
void Rect::draw() const
{
    setfillstyle(SOLID_FILL, color);
    bar(x, y, x + width, y + height);
}

void Rect::undraw() const
{
    setfillstyle(SOLID_FILL, BLACK);
    bar(x, y, x + width, y + height);
}
```

Keep it DRY. (Don't Repeat Your Self)

Follow my demo to turn the code DRY.

Code Refactoring #2: Constructors

The following constructors are repeated. Turn it to DRY code.

```
Rect::Rect(int _x, int _y, int _width, int _height, int _color)
{
    x = _x;
    y = _y;
    width = _width;
    height = _height;
    color = _color;
}

Rect::Rect(int _x, int _y, int _length, int _color)
{
    x = _x;
    y = _y;
    width = height = _length;
    color = _color;
}

Rect::Rect()
{
    x = y = 0;
    width = height = 10;
    color = WHITE;
}
```

Code Refactoring #3: Add some more methods to make the class more friendly and intuitive

For example, instead of using the name getX(), why not using the name **left()**. Then**,** we can also have **right()**

Files to edit: **rect.hpp** and **rect.cpp**.

```
class Rect
{
private:
    int x, y; // The top-left corner coordinates
    int width, height;
    int color;

public:

     ……

    int left() const;
    int top() const;
    int right() const;
    int bottom() const;
     ……
};
```

Code Refactoring #4: The main loop

```
    int dx = 50;
    while (!kbhit())
    {
        s.draw();
        delay(100);
        s.undraw();
        int x = s.getX();
        s.setX(x + dx);

        if ((s.getX() <= 0) || ((s.getX() + s.getWidth()) >= screenWidth)
)            dx = -dx;
    }
```

Files to edit: rect.hpp and rect.cpp. Add new methods named **move()** and **reverse()**
Follow along my demo for the solution.